Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения информационных технологий

Учебная дисциплина «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе №4
«Прокси-сервер»

Подготовила: Студент гр. 410902
Кухто В.С.

Проверила: Болтак С.В.

Минск 2026

Условие задания: Необходимо реализовать простой прокси-сервер, выполняющий журналирование проксируемых HTTP-запросов.

Программа должна отображать в консоли журнал с краткой информацией о проксируемых запросах (URL и код ответа).

Обязательной является поддержка HTTP, поддержка HTTPS не требуется.

Для проверки работоспособности необходимо настроить в браузере работу через прокси и попробовать загрузить ресурсы по HTTP:

http://example.com/

http://live.legendy.by:8000/legendyfm - онлайн радио, необходимо для проверки, что соединение не разрывается раньше времени.

Особенности формата URI в HTTP-запросах к прокси-серверу

В соответствии с RFC https://tools.ietf.org/html/rfc2616#section-5.1.2 тело HTTP-запроса может содержать как путь, так и URL целиком (абзац 1). При работе через прокси-сервер клиент обязан всегда передавать URL целиком (абзац 3). Также RFC предписывает, что в целях совместимости с будущими версиями HTTP, все серверы должны поддерживать оба варианта, хотя клиенты обязаны использовать полный URL только при работе через прокси-сервер (абзац 4). Как выяснилось на практике, не все серверы работают с полными URL, поэтому необходимо преобразовывать его в путь при передаче серверу назначения того, что было передано прокси-серверу от браузера.

К примеру:

Браузер -> прокси-сервер: GET http://live.legendy.by:8000/legendyfm HTTP/1.1

Прокси-сервер -> сервер назначения (live.legendy.by): GET /legendyfm HTTP/1.1

Дополнительное задание

Реализовать фильтрацию сайтов по черному списку. В конфигурационном файле для прокси-сервера задается список доменов и/или URL-адресов для блокировки.

При попытке загрузить страницу из черного списка прокси-сервер должен вернуть предопределенную страницу с адресом, доступ к которому заблокирован, и сообщением об ошибке.

Выполнение работы:

Написание было выполнено на языке Python. Разработка прокси-сервера происходила в среде Anaconda, в приложении Spyder. Запустить консольную программу можно, скачав файл proxy_server.py, bat-файл run.bat в одну папку, затем запустив файл run.bat.

Для тестирования работы программы мной была проведена настройка браузера (FireFox). На рисунке 1 можно увидеть, как именно была осуществлена настройка:

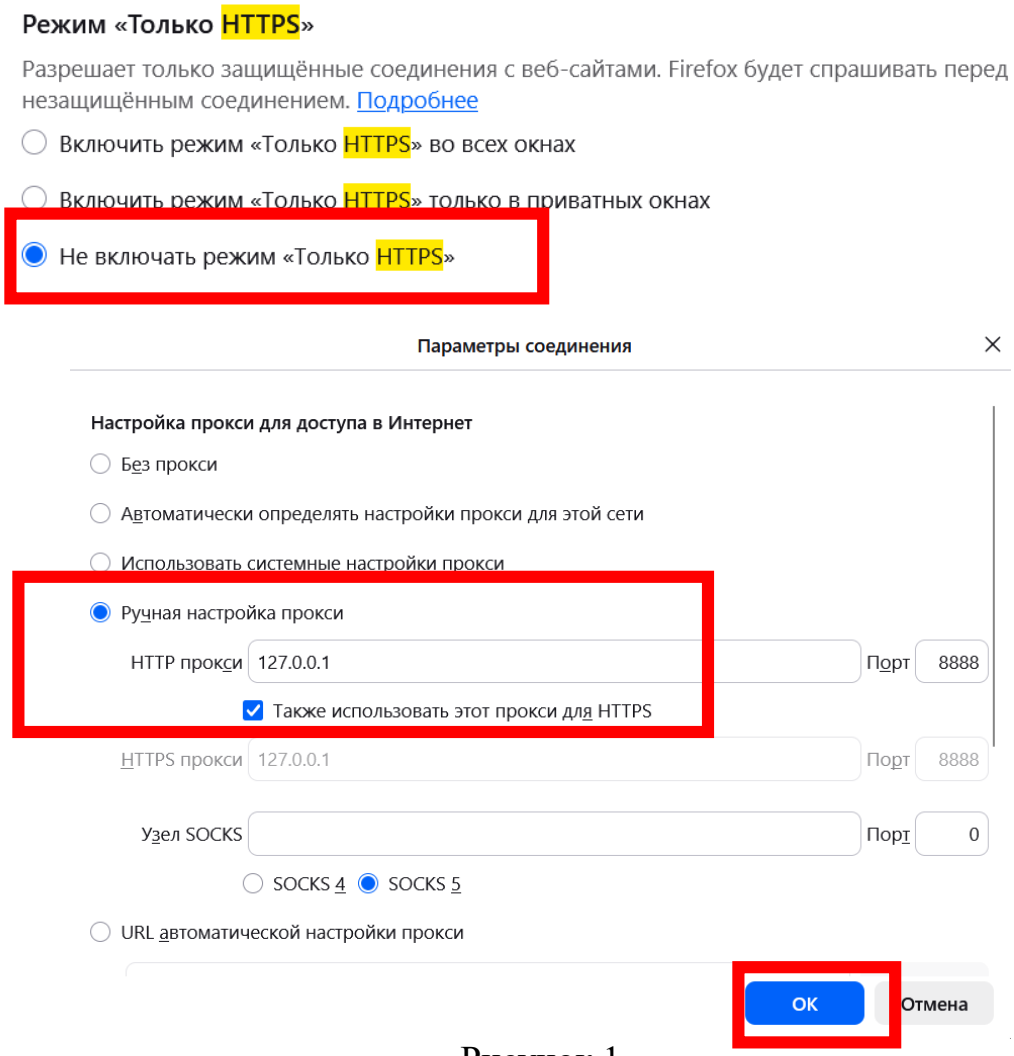


Рисунок 1

На рисунке 2 можно увидеть, как выглядит запущенный в командной строке прокси-сервер.

```
HTTP Proxy Server


Configuration:
- Listening on: 127.0.0.1:8888
- Blacklist: blacklist.txt


Proxy server started on 127.0.0.1:8888
Blacklist contains 1 entries
Press Ctrl+C to stop
```

Рисунок 2

Как видно на рисунке 3, был создан экземпляр класса ProxyServer с параметрами host='127.0.0.1', port=8888 и blacklist_file='blacklist.txt'.

```
proxy = ProxyServer(host='127.0.0.1', port=8888, blacklist_file='blacklist.txt')
proxy.start()
```

Рисунок 3

На рисунке 4.1 можно увидеть, устанавливается соединение браузера с прокси-сервером (3-этапное рукопожатие по tcp). Браузер инициирует TCP-соединение с прокси-сервером, отправляя SYN-пакет с начальным порядковым номером 0. Прокси-сервер отвечает, подтверждая получение SYN (Ack=1) и отправляя свой SYN. Браузер подтверждает получение SYN-ACK, соединение установлено. Соединение браузер - прокси использует loopback-интерфейс (127.0.0.1), так как оба процесса запущены на одном компьютере. Этот трафик не выходит за пределы компьютера, поэтому в Wireshark только адрес 127.0.0.1:

21	0.000888	127.0.0.1	127.0.0.1	TCP	56	49926 → 8888 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256
22	0.000951	127.0.0.1	127.0.0.1	TCP	56	8888 → 49926 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65
23	0.000982	127.0.0.1	127.0.0.1	TCP	44	49926 → 8888 [ACK] Seq=1 Ack=1 Win=65280 Len=0

Рисунок 4.1

Как видно на рисунке 4, в браузере была открыта страница по адресу http://live.legendy.by:8000/legendyfm.

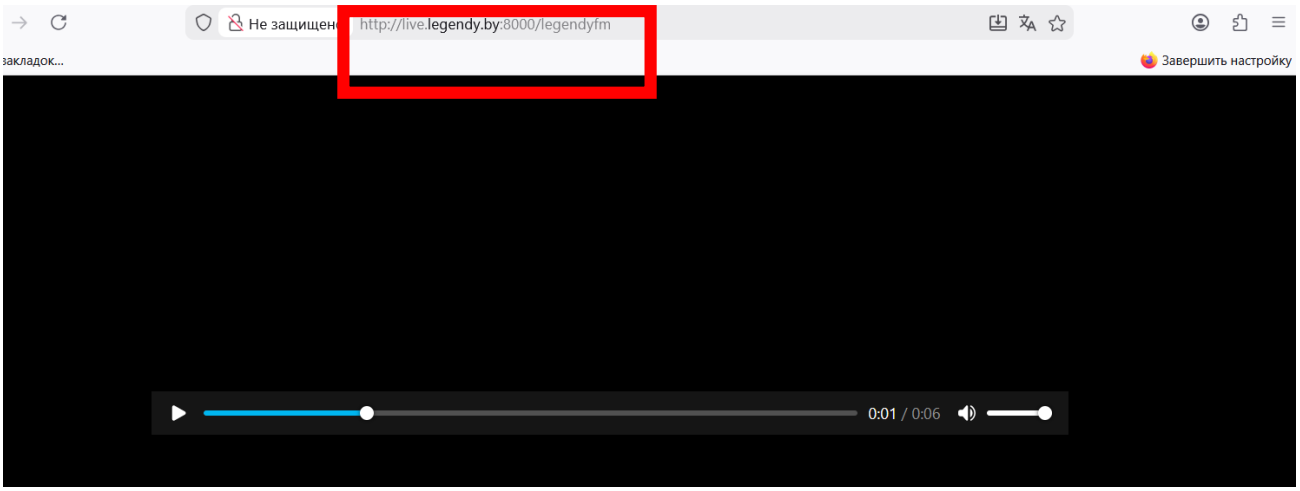


Рисунок 4

На рисунке 5 видно, что вывелось в консоли. Код 200 означает успешную загрузку страницы. В каждой записи есть дата и время, URL, код ответа.

```
[2026-03-23 22:09:57] http://live.legendy.by:8000/legendyfm - 200 OK
```

Рисунок 5

На рисунке 7 можно увидеть захваченные пакеты в программе вайршарк (фильтр отображения http). Это запрос от браузера к прокси серверу. Соединение браузер - прокси использует loopback-интерфейс (127.0.0.1), так как оба процесса запущены на одном компьютере (как прокси пересылает запрос серверу и получает ответ от него будет рассмотрено далее на другом примере с другим интерфейсом в вайршарке).

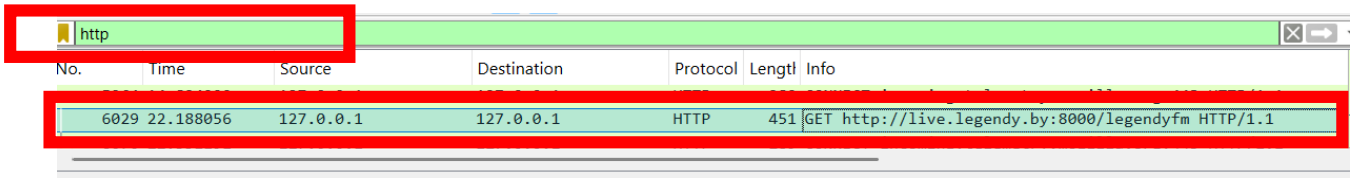


Рисунок 7

На рисунке 8 можно увидеть содержимое http-запроса. Request Method: GET - запрос на получение ресурса. Request URI: http://live.legendy.by:8000/legendyfm в соответствии с RFC, через прокси

передается полный URL. Host: live.legendy.by - имя сервера назначения. Идентификатор браузера User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0) Gecko/20100101 Firefox/148.0\r\n.

```
Hypertext Transfer Protocol
  GET http://live.legendy.by:8000/legendyfm HTTP/1.1\r\n
    Request Method: GET
    Request URI: http://live.legendy.by:8000/legendyfm
    Request Version: HTTP/1.1
  Host: live.legendy.by:8000\r\n
  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0) Gecko/20100101 Firefox
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
  Accept-Language: ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7\r\n
  Accept-Encoding: gzip, deflate\r\n
  Connection: keep-alive\r\n
  Upgrade-Insecure-Requests: 1\r\n
  Priority: u=0, i\r\n
  \r\n
  [Full request URI: http://live.legendy.by:8000/legendyfm]
```

Рисунок 8

Ответ сервера есть, просто он не отображается как отдельный HTTP-пакет. Ответ сервера представляет собой непрерывный аудиопоток. Факт успешного установления соединения подтверждается логом прокси-сервера (200) и воспроизведением аудио в браузере (далее в отчете будут проанализированы ответы в вайршарке).

Дополнительно было выполнено задание для фильтрации сайтов по черному списку. На рисунке 9 можно увидеть, какие сайты находятся в моем черном списке:

```
blacklist.txt
1   tracker.com
2   openbsd.org
3   example.com
```

Рисунок 9

При попытке открытия сайта tracker.com (из черного списка), в браузере открывается следующая страница (рисунок 10):

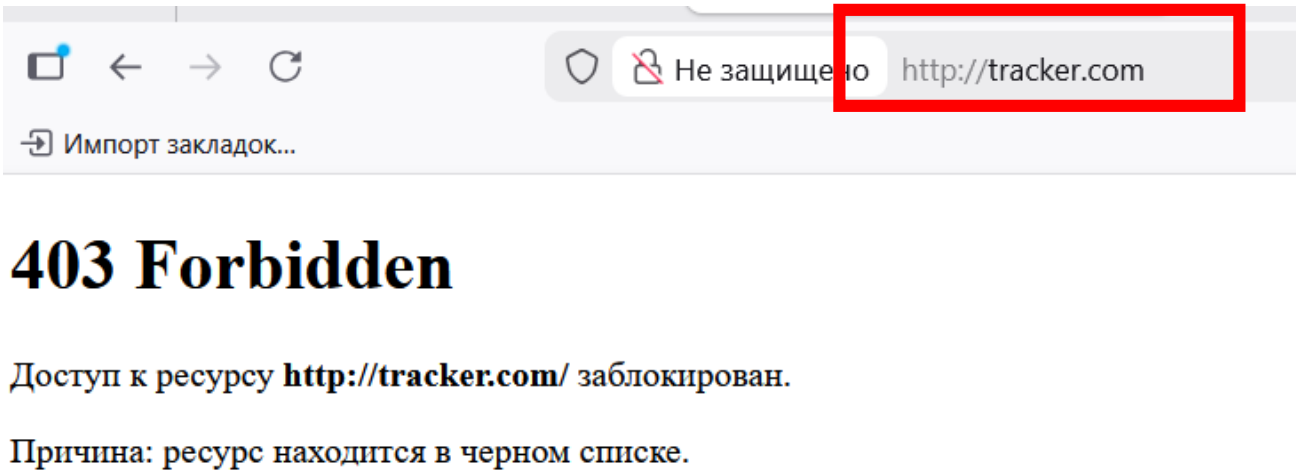


Рисунок 10

В консоли при этом можно увидеть следующее (рисунок 11). Запрос получил код 403. Прокси не обращается к целевому серверу, а сразу возвращает страницу блокировки:

```
[2026-03-23 22:09:57] http://live.legendy.by:8000/legendyfm - 200 OK
[2026-03-23 22:12:45] http://tracker.com/ - 403 Forbidden
```

Рисунок 11

В вайршарке при этом можно увидеть следующие пакеты (рисунок 12):

127.0.0.1	127.0.0.1	HTTP	424 GET http://tracker.com/ HTTP/1.1
127.0.0.1	127.0.0.1	HTTP	44 HTTP/1.1 403 Forbidden (text/html)

Рисунок 12

При попытке обращения к ресурсу, указанному в черном списке (в данном случае http://tracker.com/), прокси-сервер не устанавливает соединение с целевым сервером, а немедленно возвращает клиенту HTML-страницу с кодом состояния HTTP 403 Forbidden. В вайршарке можно увидеть html-разметку этой страницы (рисунок 13):

```
Line-based text data: text/html (9 lines)
    <!DOCTYPE html>\n
    <html>\n
    <head><title>Доступ заблокирован</title></head>\n
    <body>\n
        <h1>403 Forbidden</h1>\n
        <p>Доступ к ресурсу <strong>http://tracker.com/</strong> заблокирован.</p>\n
        <p>Причина: ресурс находится в черном списке.</p>\n
    </body>\n
    </html>
```

Рисунок 13

Для разбора ответа в вайршарке была открыта еще одна страница по адресу: http://www.httpvshttps.com/. Ее внешний вид можно увидеть на рисунке 14.

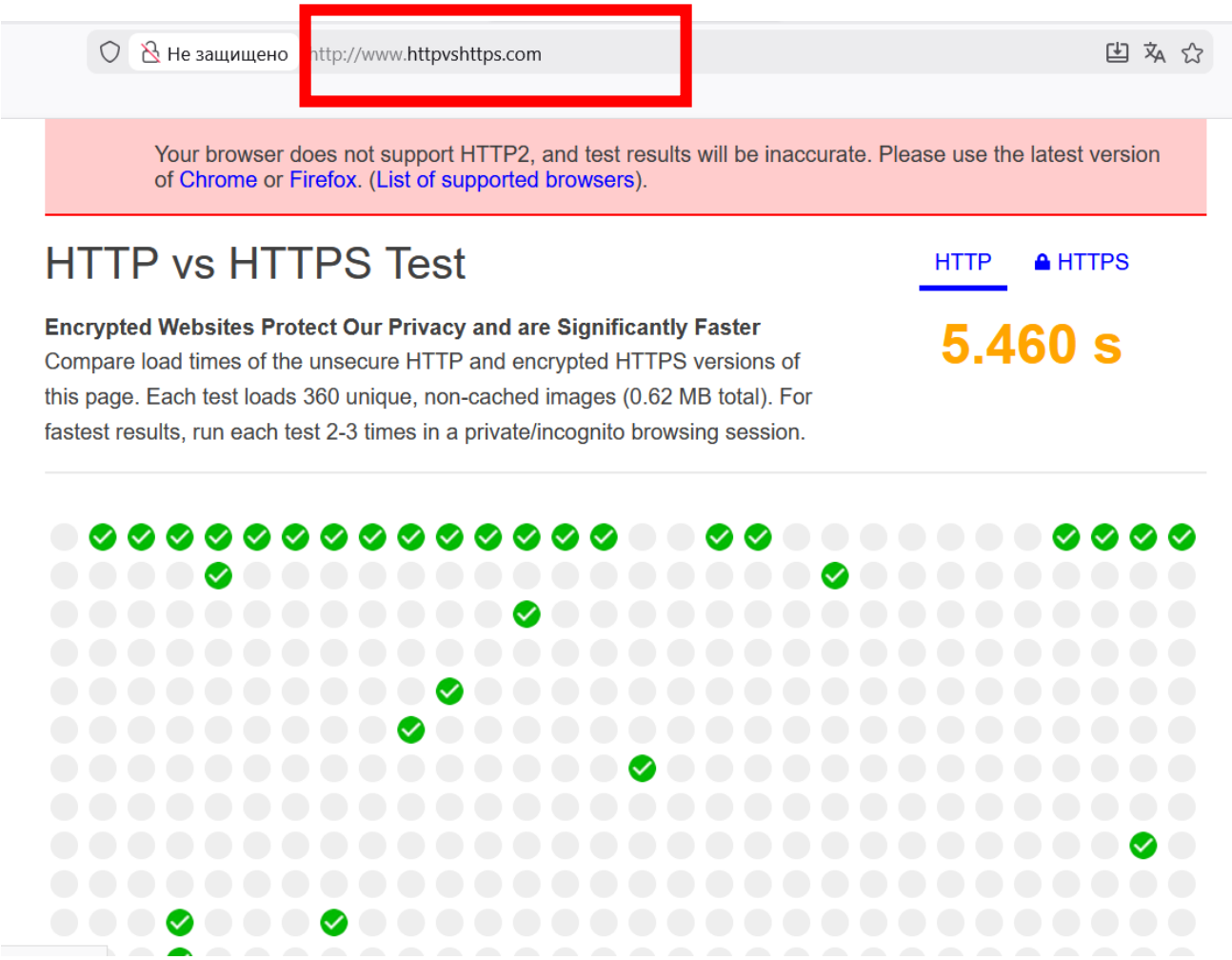


Рисунок 14

Большое число ответов можно увидеть на рисунке 15:

```
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_21 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_20 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_22 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_41 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_70 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_69 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_31 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_312 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_315 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_68 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_311 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_313 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_314 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_318 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_316 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_319 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_322 - 200 OK
[2026-03-23 22:17:01] http://www.httpvshttps.com/check.png?17742934108545_323 - 200 OK
```

Рисунок 15

При загрузке страницы http://www.httpvshttps.com/ браузер автоматически запрашивает все изображения, встроенные в HTML, что видно в логах.

Проанализирую один из таких ответов (рисунок 16):

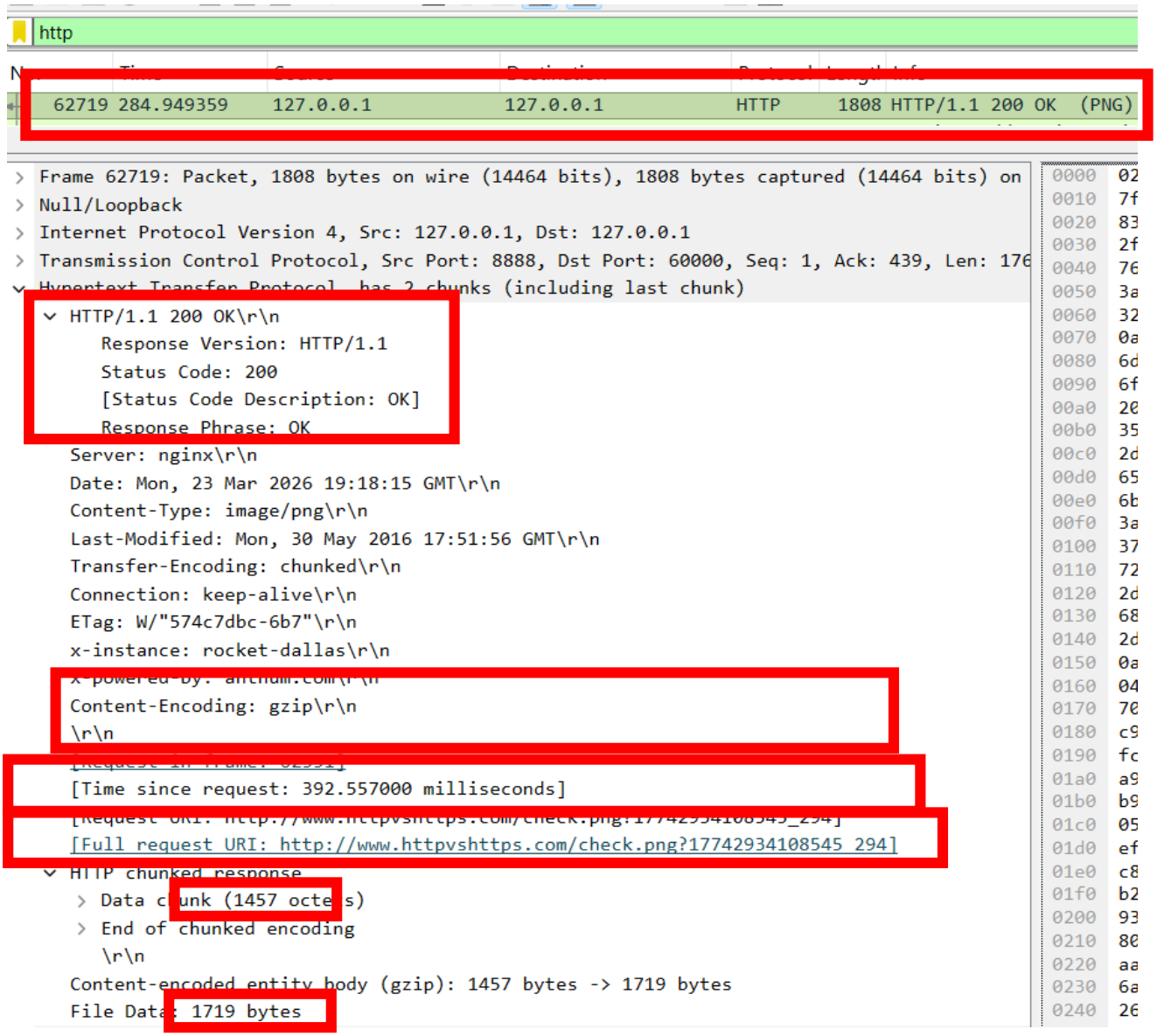


Рисунок 16

В программе вайршарк был зафиксирован HTTP-ответ с кодом 200, который сервер отправил через прокси-сервер. Тип содержимого - image/png. Стартовая строка ответа: HTTP/1.1 200 OK. Версия протокола HTTP/1.1, код состояния 200 означает успешную обработку запроса, текстовая фраза OK подтверждает, что сервер вернул запрошенный ресурс. Заголовок Transfer-Encoding: chunked указывает, что данные передаются частями, а Content-Encoding: gzip показывает, что содержимое сжато (1457 байт сжатых данных распаковываются в 1719 байт). Заголовок Connection: keep-alive означает, что сервер предлагает оставить TCP-соединение открытым для последующих запросов. Заголовок Content-Type: image/png определяет тип содержимого как изображение в формате PNG, чтобы браузер знал, как правильно отобразить

полученные данные. В информации о пакете также указан запрос, на который был получен этот ответ: Request URI: /check.png?17742934108545_294 и полный URI запроса: http://www.httpvshttps.com/check.png?17742934108554_294.

Увидеть, как прокси шлет серверу запрос и получает ответ можно увидеть на рисунке 16.1 (выбран другой интерфейс в вайршарке). 192.168.1.184 ip адрес моего компьютера, 45.33.7.16 - сервера:

192.168.1.184	45.33.7.16	HTTP	464 GET /check.png?1774900525638_96 HTTP/1.1
45.33.7.16	192.168.1.184	HTTP	438 HTTP/1.1 200 OK (PNG)

Рисунок 16.1

В моей программе реализована многопоточность. Для каждого нового клиента создается отдельный поток. Потоки работают параллельно и не блокируют друг друга (соответствующая часть кода на рисунке 17.1).

```
while True:
    client_socket, address = server_socket.accept()
    thread = threading.Thread(target=self.handle_client, args=(client_socket, address))
    thread.daemon = True
    thread.start()
```

Рисунок 17.1

Корректную работу можно проверить, открыв браузер в двух окнах. Я попробовала получить доступ к страницам из черного списка (рисунок 17.2):

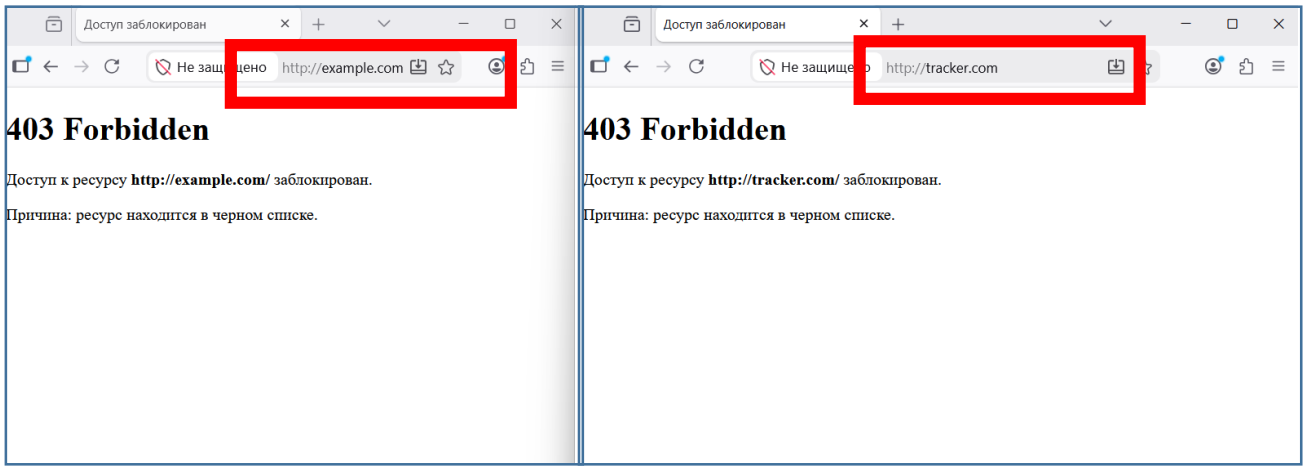


Рисунок 17.2

В логах можно увидеть соответствующие записи (рисунок 18):

В 1 окне

```
[2026-03-30 16:04:26] http://example.com/ - 403 Forbidden
[2026-03-30 16:04:41] http://tracker.com/ - 403 Forbidden
```

Во 2 окне

Рисунок 18

В вайршарке можно увидеть соответствующие пакеты (рисунок 19):

127.0.0.1	127.0.0.1	HTTP	424 GET http://example.com/ HTTP/1.1
127.0.0.1	127.0.0.1	HTTP	44 HTTP/1.1 403 Forbidden (text/html)
127.0.0.1	127.0.0.1	HTTP	424 GET http://tracker.com/ HTTP/1.1
127.0.0.1	127.0.0.1	HTTP	44 HTTP/1.1 403 Forbidden (text/html)

Рисунок 19

Также протестировать работу прокси-сервера можно с помощью сервиса httpbin.org. Он создан для тестирования HTTP-запросов. Далее будут приведены примеры логов в командной строке после открытия соответствующих разделов сайта.

Рисунок 20 http://httpbin.org/status/200 (успешный ответ).

```
[2026-03-30 18:25:56] http://httpbin.org/get - 200 OK
```

Рисунок 20

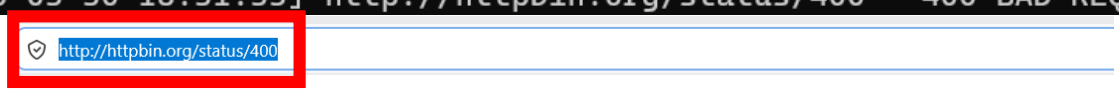
Рисунок 21 http://httpbin.org/status/302 (временное перенаправление).

```
[2026-03-30 18:30:36] http://httpbin.org/status/302 - 302 FOUND
```

Рисунок 21

Рисунок 22 http://httpbin.org/status/400 (неправильный запрос).

```
[2026-03-30 18:31:33] http://httpbin.org/status/400 - 400 BAD REQUEST
```



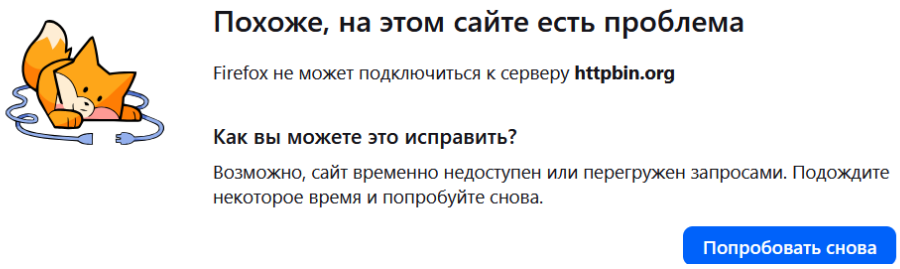


Рисунок 22

Рисунок 23 http://httpbin.org/status/403 (запрещенный ресурс). Ранее для сайтов, написанных в моем запрещенном списке, был аналогичный лог.

```
[2026-03-30 18:33:16] http://httpbin.org/status/403 - 403 FORBIDDEN
```

Рисунок 23

Рисунок 24 http://httpbin.org/status/500 (ошибка сервера).

```
[2026-03-30 18:34:37] http://httpbin.org/status/500 - 500 INTERNAL SERVER ERROR
```

Рисунок 24

Как видно на скриншотах выше, программа работает корректно.